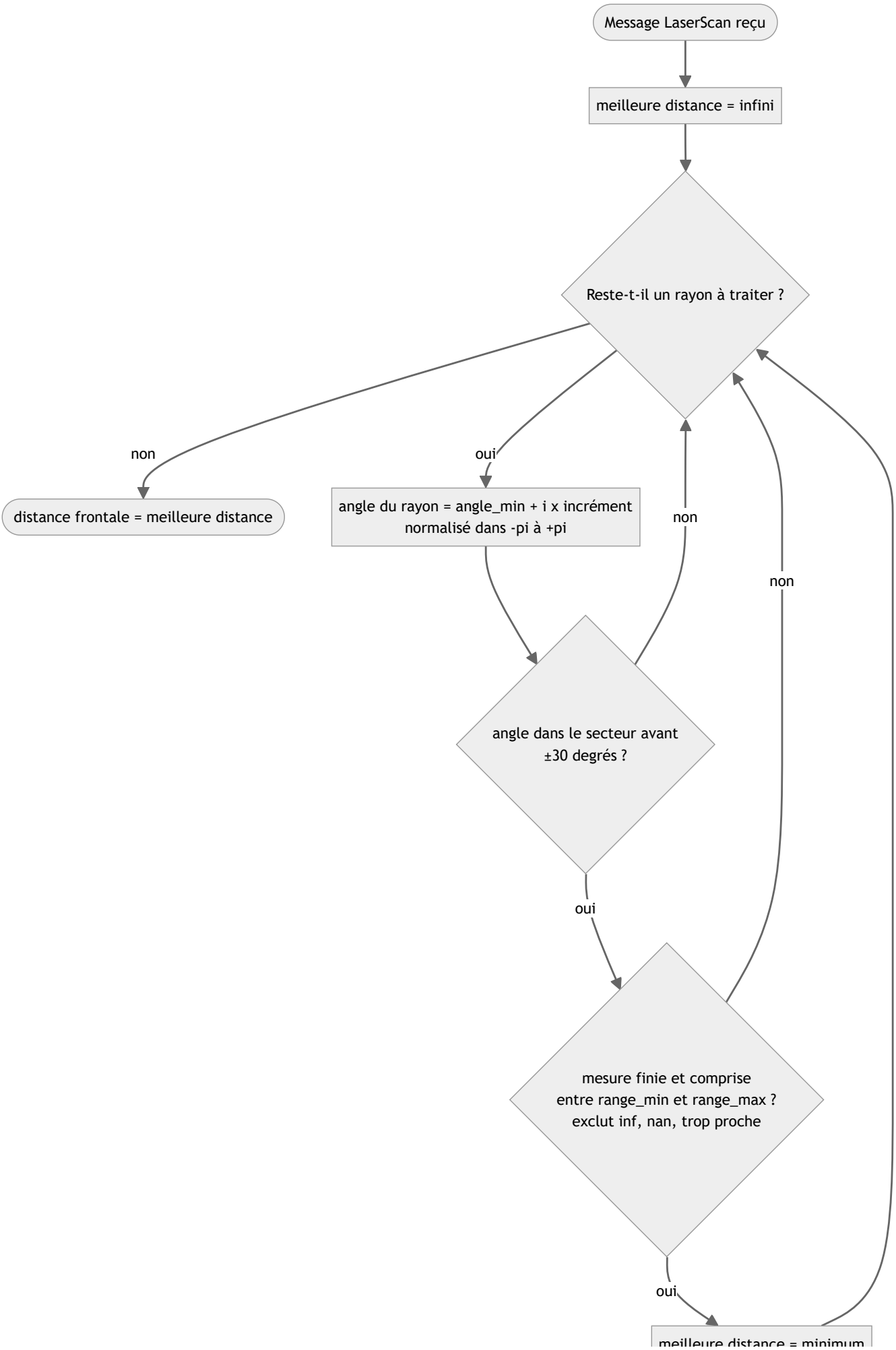
Algorigrammes

Les slides ne contiennent pas de code : ces algorigrammes les remplacent. Fidèles à `random_logic.py`, `random_walker_node.py`, `planning.py`, `coverage_planner_node.py`, `coverage_executor_node.py` et `coverage_tracker_node.py`. Le planner (stratégie B) est décrit ici au niveau de `planning.py`, puis du nœud `coverage_planner`. L'executor (section 3) enchaîne l'envoi du trajet à Nav2.

---

1. random_walker (stratégie A)

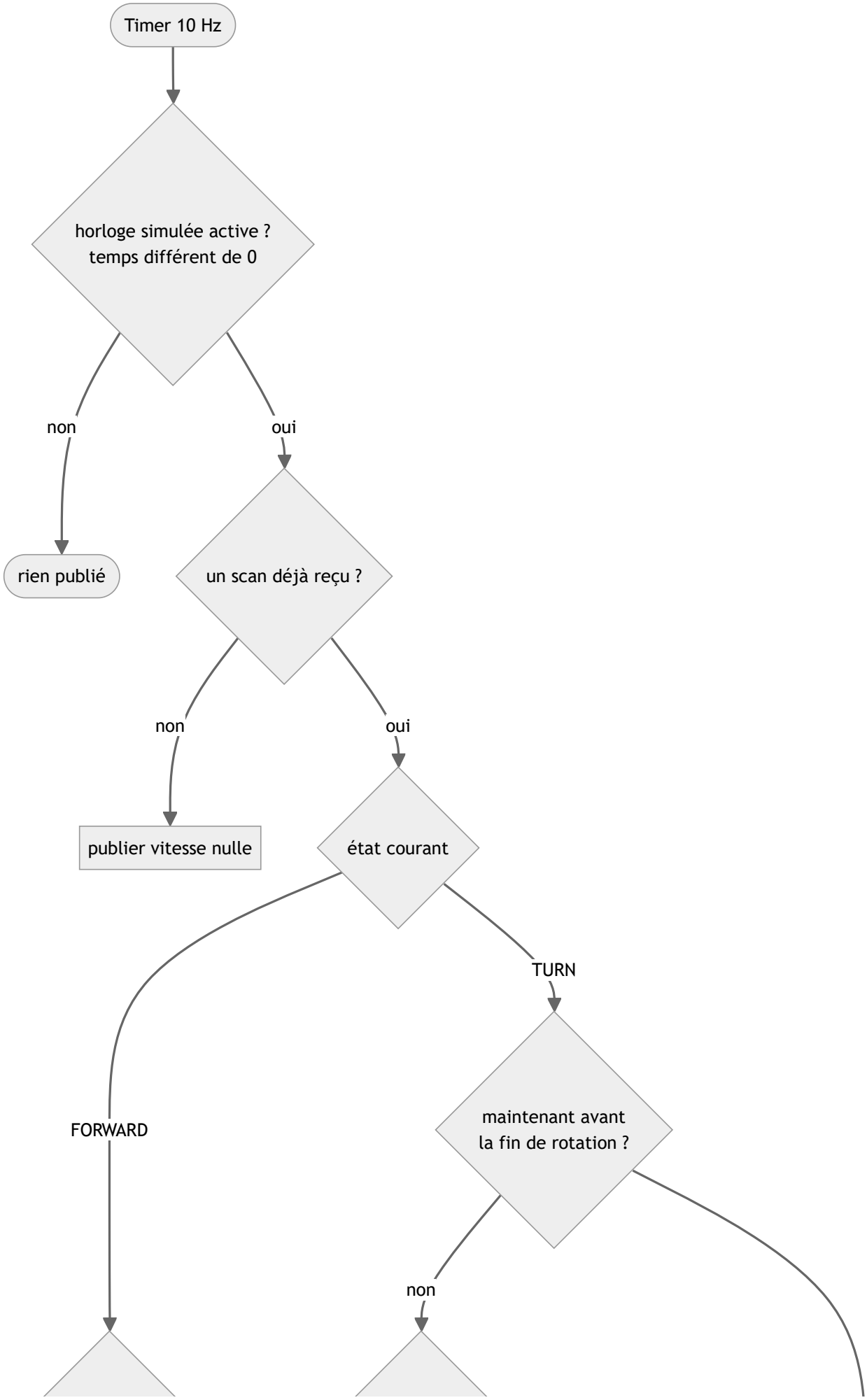
1.1 Distance frontale (à chaque message `/scan`)

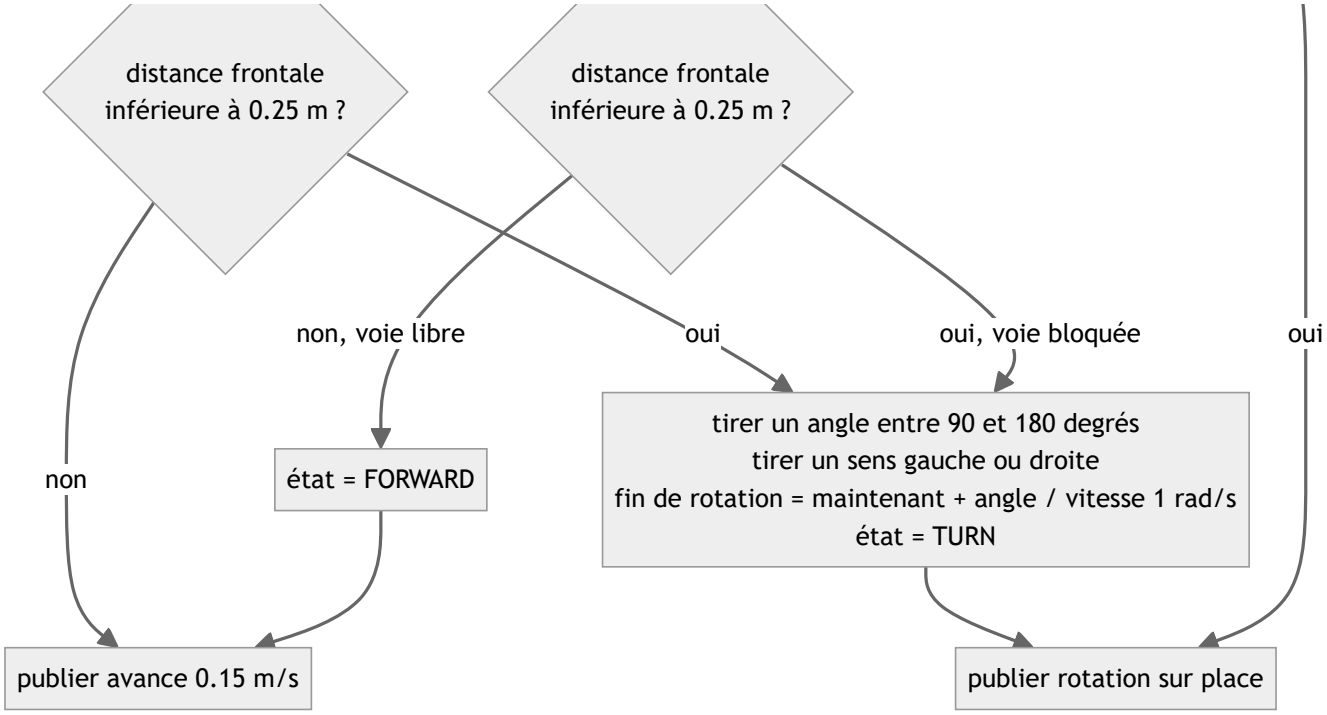

Message LaserScan reçu
meilleure distance = infini
Reste-t-il un rayon à traiter ?
non
oui
distance frontale = meilleure distance
angle du rayon = angle_min + i x incrément
normalisé dans -pi à +pi
non
angle dans le secteur avant
±30 degrés ?
non
oui
mesure finie et comprise
entre range_min et range_max ?
exclut inf, nan, trop proche
oui
meilleure distance = minimum

meilleure distance = minimum
entre elle et la mesure

Le scan du TurtleBot3 va de 0 à 2π : l'avant est au début et à la fin du tableau. La normalisation des angles dans [-π, π] gère ce cas. S'il n'y a aucune mesure valide, la distance vaut l'infini (voie libre).

1.2 Machine à états (timer à 10 Hz)


Timer 10 Hz
horloge simulée active ?
temps différent de 0
non
oui
rien publié
un scan déjà reçu ?
non
oui
publier vitesse nulle
état courant
TURN
FORWARD
maintenant avant
la fin de rotation ?
non



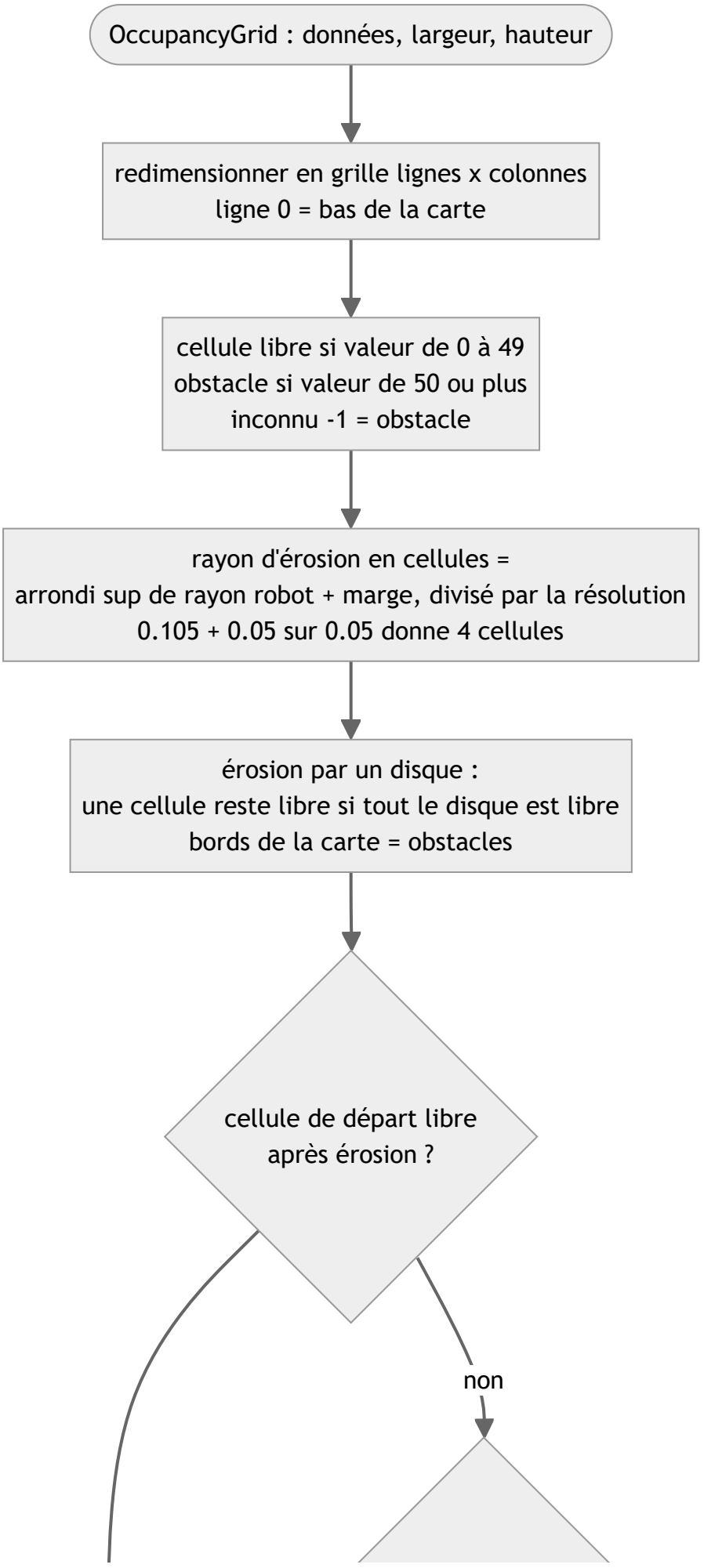
- Deux états seulement : `FORWARD` et `TURN`.
- Le hasard vient d'un générateur local initialisé par `seed` : même graine, même suite de rotations.
- La rotation est temporisée (durée = angle / vitesse angulaire), sans odométrie. L'angle réel peut différer légèrement de l'angle tiré : sans importance pour une marche aléatoire.
- Si la voie est encore bloquée à la fin de la rotation, un nouveau tirage est fait.
- La commande est publiée sur `/cmd_vel` (`geometry_msgs/Twist`) à chaque tick.

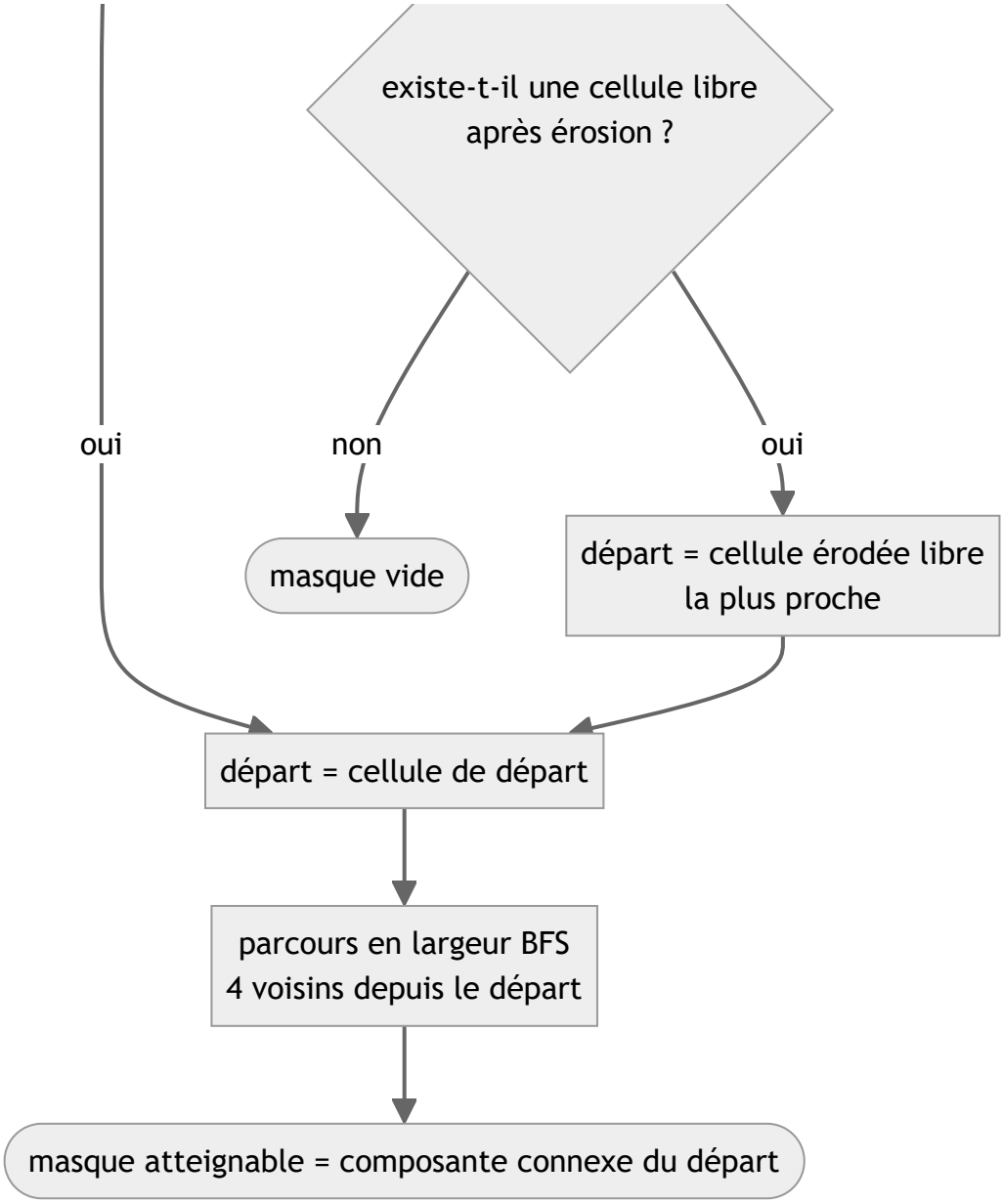
---

2. Masque atteignable et boustrophédon (`planning.py`)

2.1 Masque atteignable

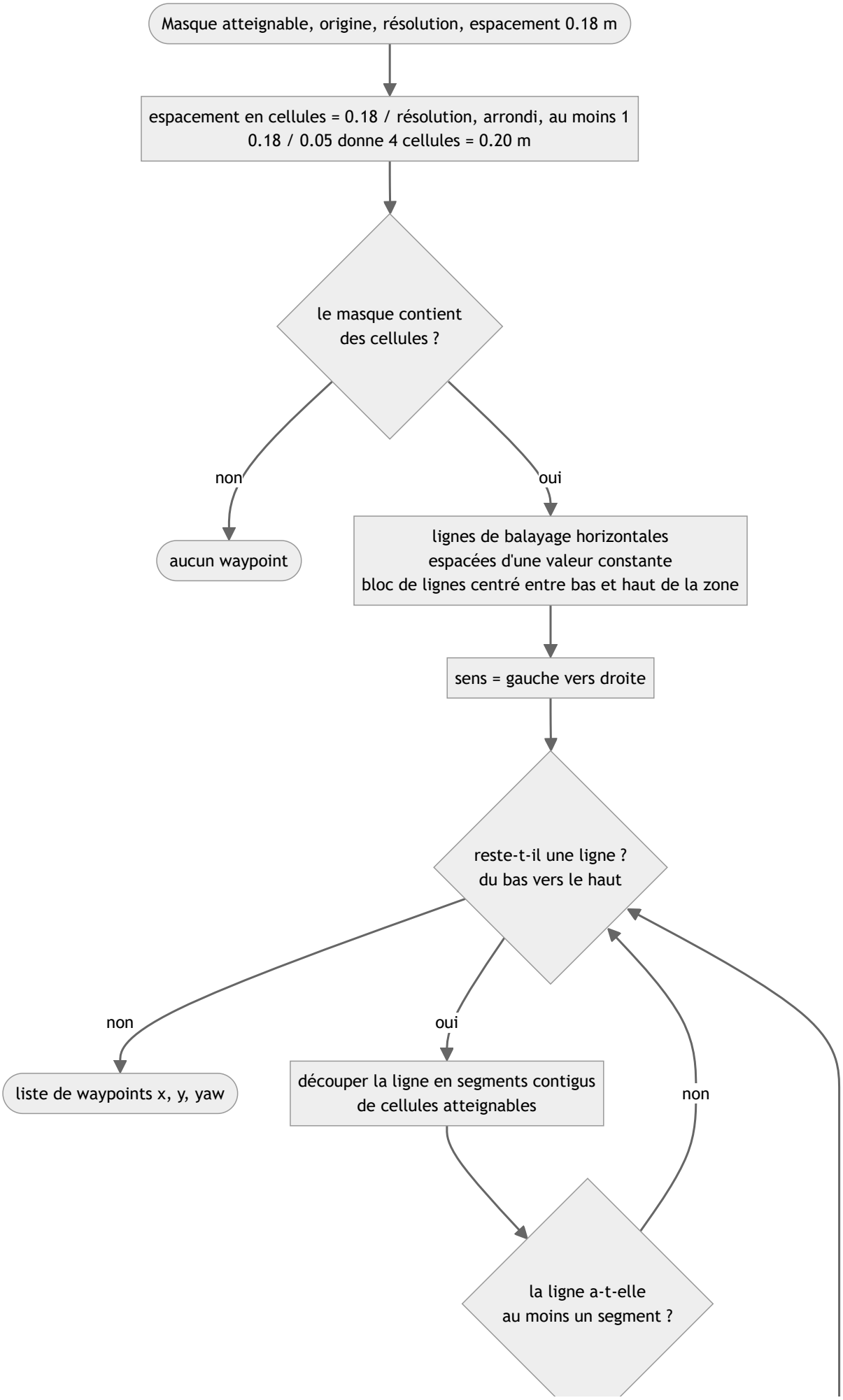
Utilisé par le planner (tracé) et par le tracker (dénominateur du pourcentage).

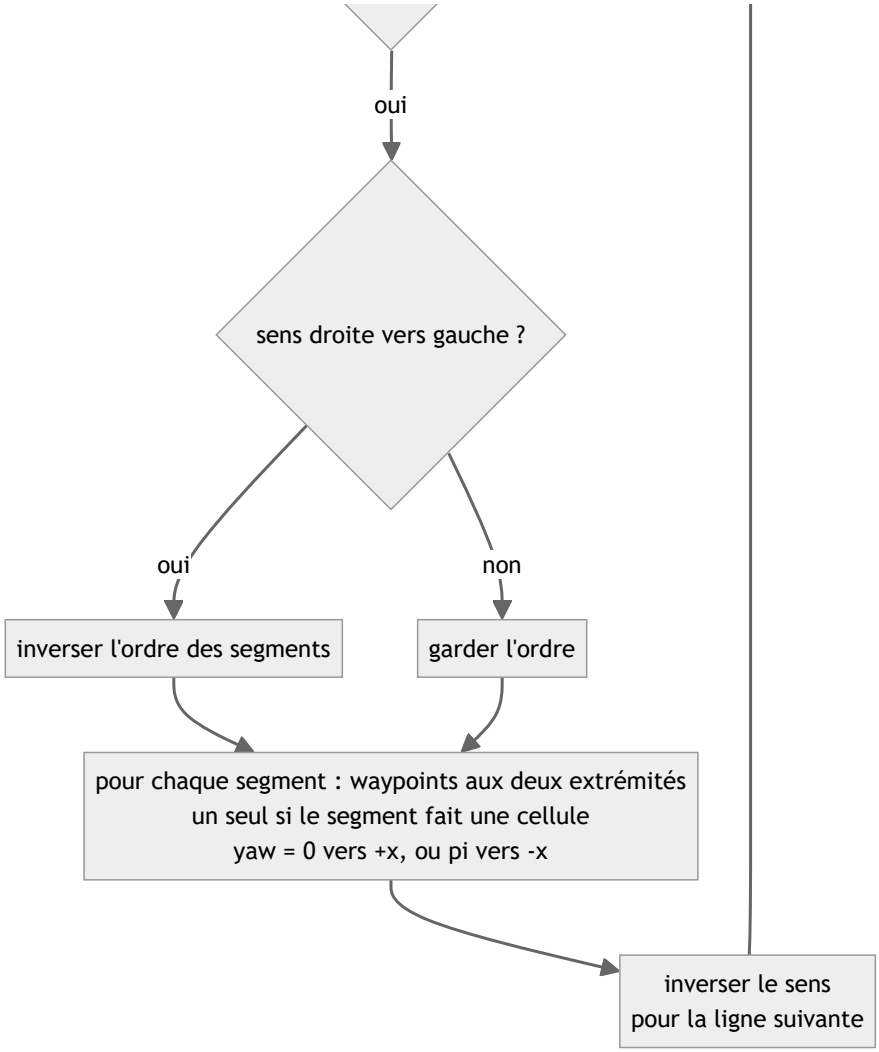

OccupancyGrid : données, largeur, hauteur
redimensionner en grille lignes x colonnes
ligne 0 = bas de la carte
cellule libre si valeur de 0 à 49
obstacle si valeur de 50 ou plus
inconnu -1 = obstacle
rayon d'érosion en cellules =
arrondi sup de rayon robot + marge, divisé par la résolution
0.105 + 0.05 sur 0.05 donne 4 cellules
érosion par un disque :
une cellule reste libre si tout le disque est libre
bords de la carte = obstacles
cellule de départ libre
après érosion ?
non



- L'inconnu est traité comme obstacle : on ne nettoie pas ce qu'on ne connaît pas.
- L'érosion garantit que le centre du robot reste à distance des murs.
- La composante connexe élimine les zones libres inaccessibles depuis le départ.
- Sur la carte `world` : 5564 cellules atteignables, soit 13.9 m^2.

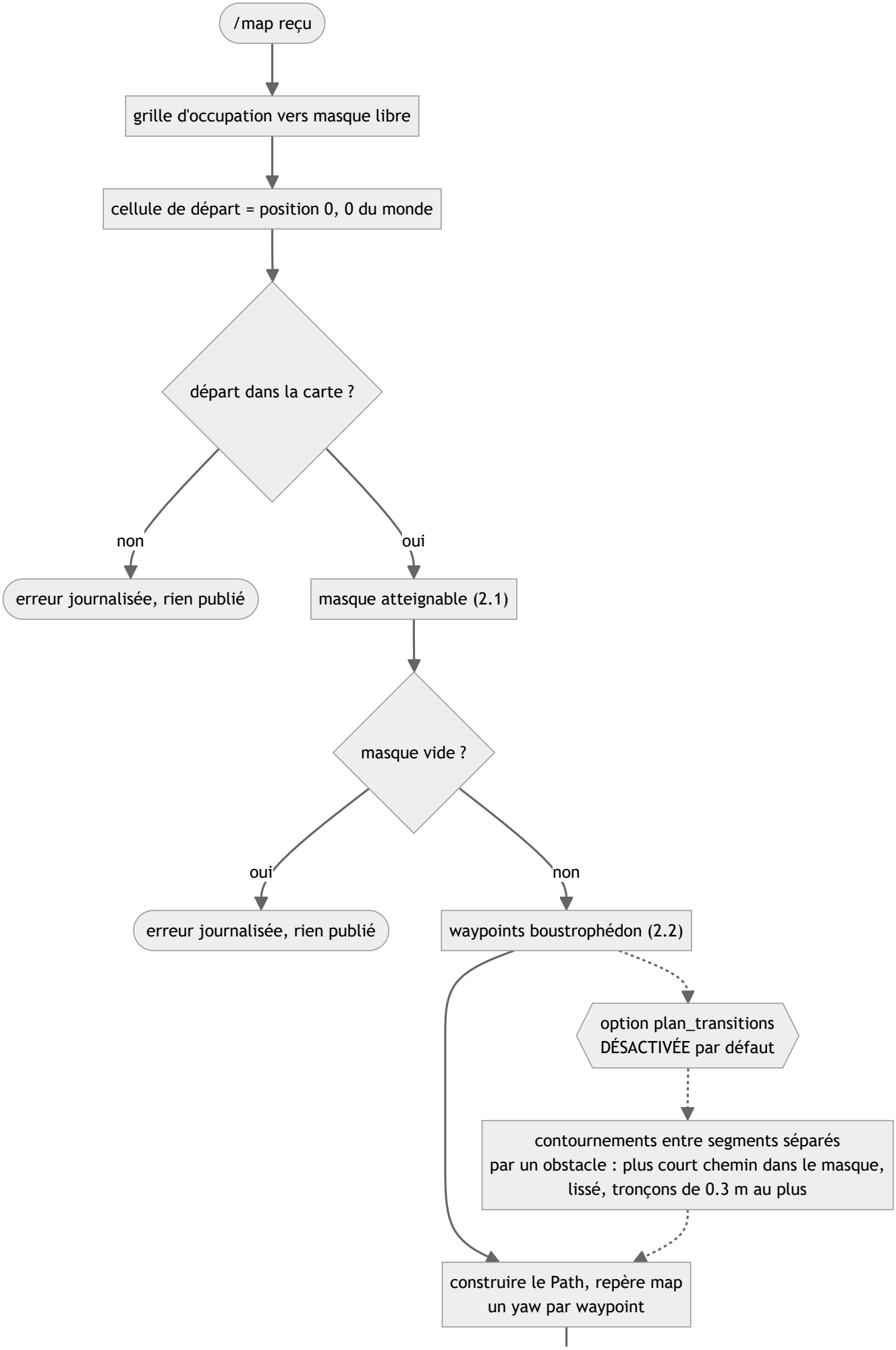
2.2 Balayage boustrophédon

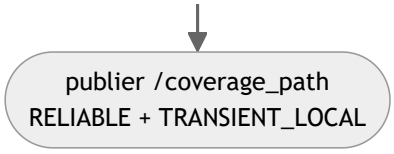

Masque atteignable, origine, résolution, espacement 0.18 m
espacement en cellules = 0.18 / résolution, arrondi, au moins 1
0.18 / 0.05 donne 4 cellules = 0.20 m
le masque contient
des cellules ?
non
oui
aucun waypoint
lignes de balayage horizontales
espacées d'une valeur constante
bloc de lignes centré entre bas et haut de la zone
sens = gauche vers droite
reste-t-il une ligne ?
du bas vers le haut
non
oui
liste de waypoints x, y, yaw
découper la ligne en segments contigus
de cellules atteignables
non
la ligne a-t-elle
au moins un segment ?



- Les lignes sont centrées : l'excédent est réparti moitié en bas, moitié en haut. L'espacement reste constant et la bande du haut est couverte (correction du jour 1).
- Les waypoints sont les extrémités des segments seulement, orientés dans le sens de passage.
- Une ligne vide ne change pas le sens.
- Limite assumée : ce n'est pas une décomposition cellulaire de Choset. Quand un obstacle coupe une ligne, les segments sont enchaînés dans l'ordre. Le contournement est laissé au planificateur de Nav2.
- Sur la carte `world` : 112 waypoints.

2.3 Nœud `coverage_planner` (à chaque `/map` reçu)


/map reçu
grille d'occupation vers masque libre
cellule de départ = position 0, 0 du monde
départ dans la carte ?
non
oui
erreur journalisée, rien publié
masque atteignable (2.1)
masque vide ?
oui
non
erreur journalisée, rien publié
waypoints boustrophédon (2.2)
option plan_transitions
DÉSACTIVÉE par défaut
contournements entre segments séparés
par un obstacle : plus court chemin dans le masque,
lissé, tronçons de 0.3 m au plus
construire le Path, repère map
un yaw par waypoint

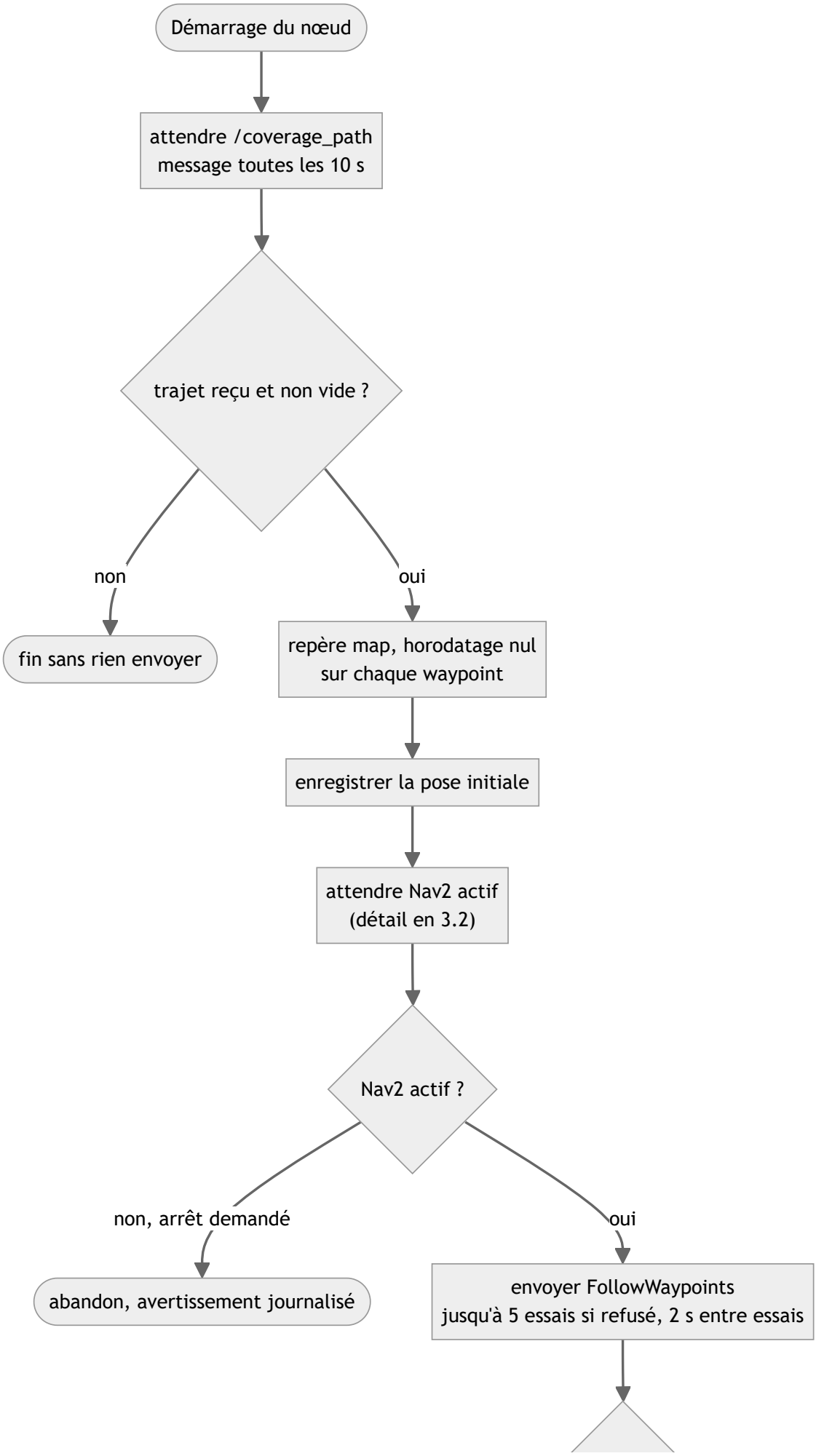


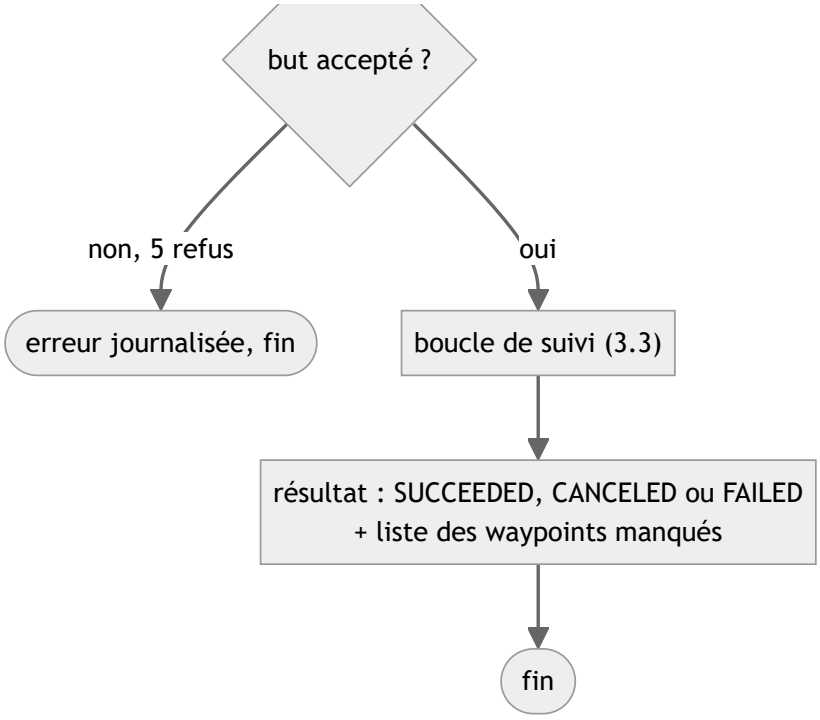
- Une seule publication par carte reçue. Le QoS TRANSIENT_LOCAL permet à l'executor, lancé en même temps, de recevoir le trajet même s'il s'abonne après.
- Pointillés : `plan_transitions` est une option de launch, désactivée. Essai T4 négatif : 112 devient 245 waypoints, mais la couverture à 300 s tombe à 22.4 % contre 28.5 % (Nav2 avec Regulated Pure Pursuit contourne déjà les obstacles ; les points intermédiaires multiplient arrêts et rotations).
- Le code est conservé et testé (8 tests unitaires) : une impasse documentée.

---

3. coverage_executor (stratégie B)

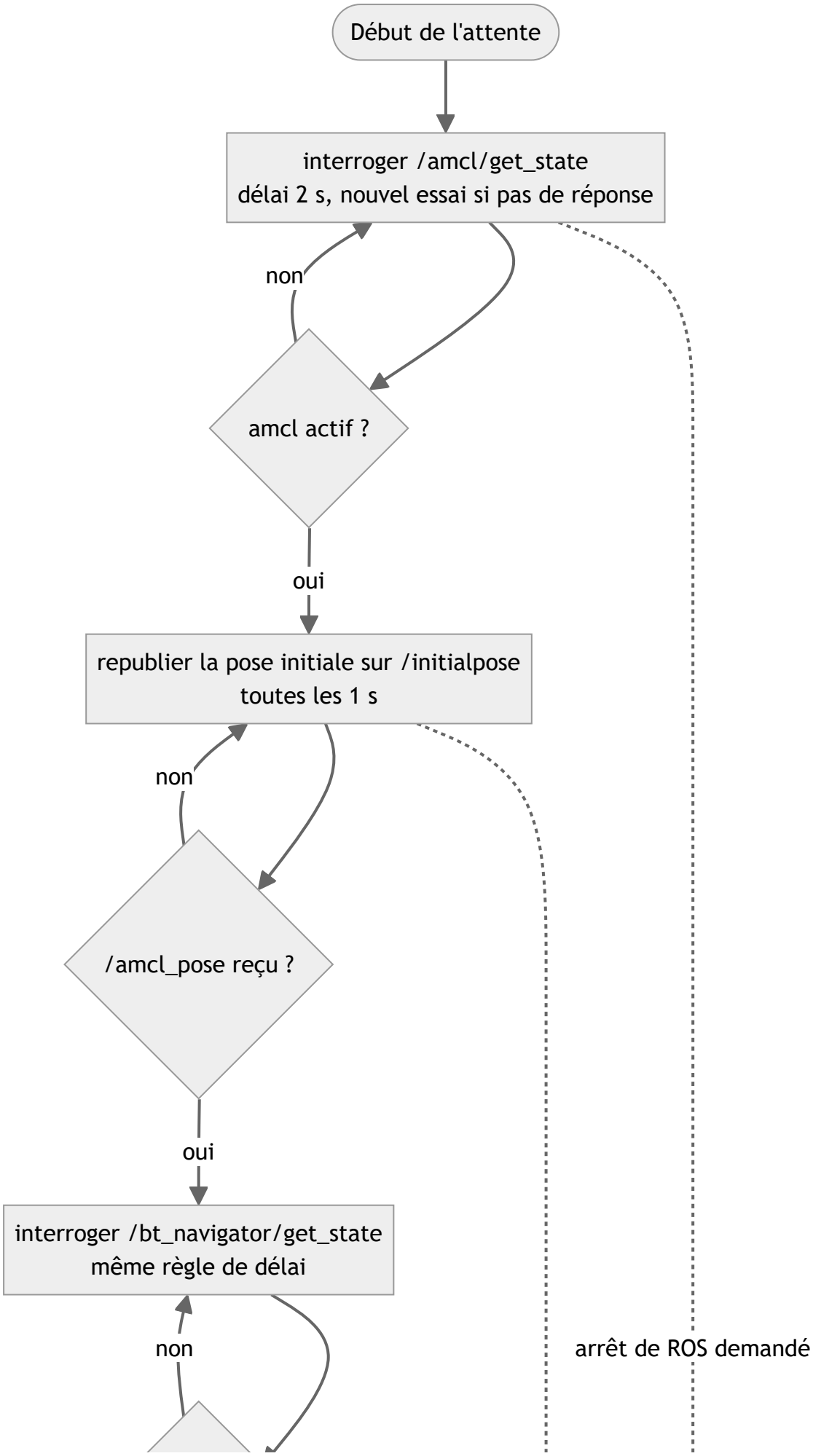
3.1 Déroulement général

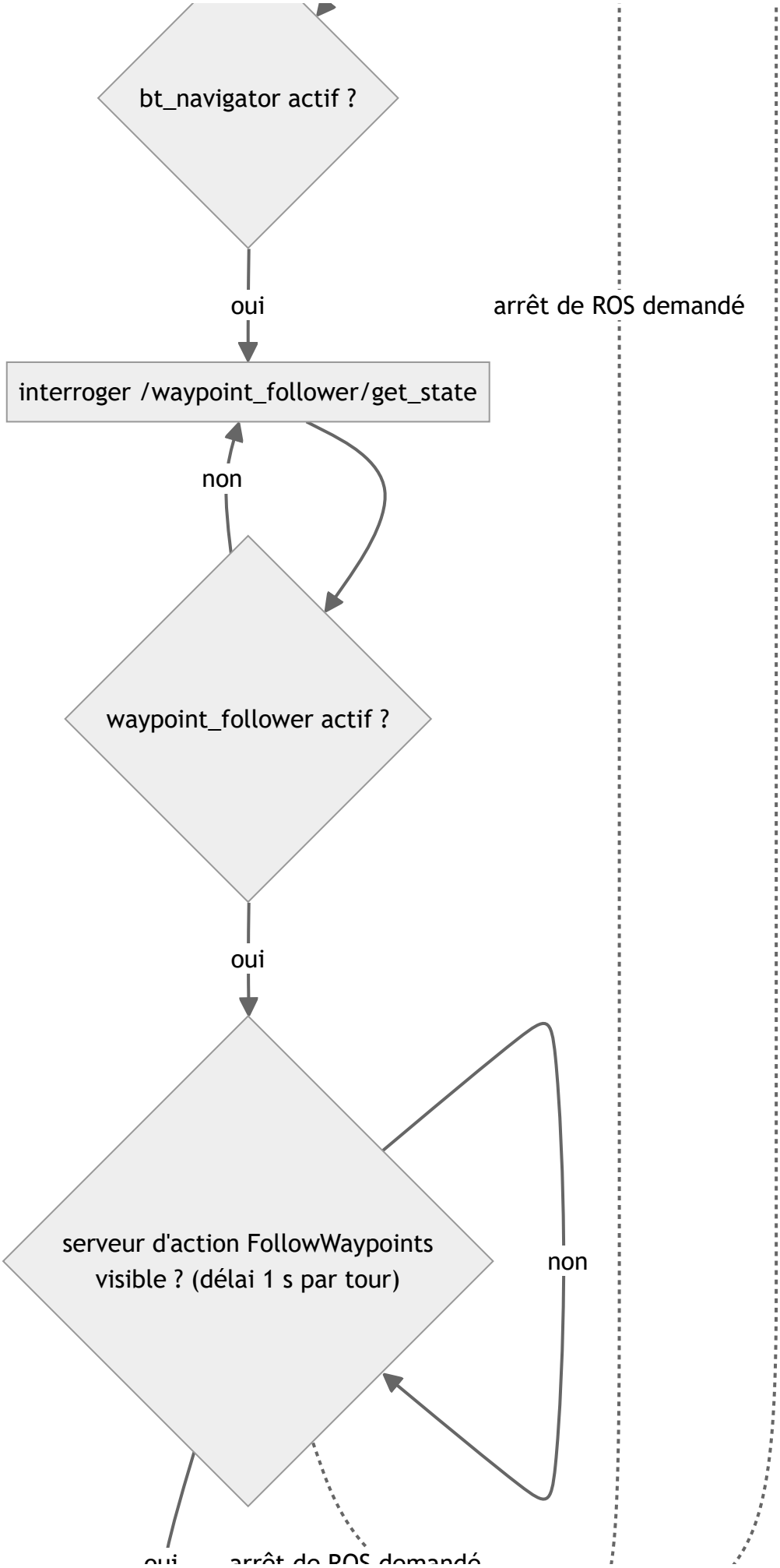

Démarrage du nœud
attendre /coverage_path
message toutes les 10 s
trajet reçu et non vide ?
non
oui
fin sans rien envoyer
repère map, horodatage nul
sur chaque waypoint
enregistrer la pose initiale
attendre Nav2 actif
(détail en 3.2)
Nav2 actif ?
non, arrêt demandé
oui
abandon, avertissement journalisé
envoyer FollowWaypoints
jusqu'à 5 essais si refusé, 2 s entre essais

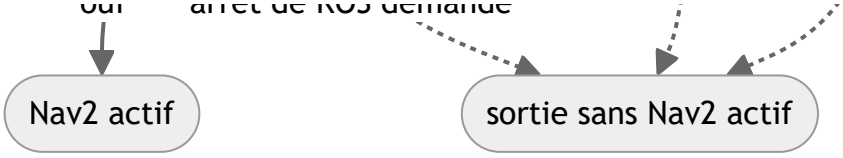


3.2 Attente de Nav2 actif

Attente maison, qui remplace `waitUntilNav2Active` de `BasicNavigator`. Aucune attente n'est sans délai : chaque interrogation du service `GetState` (`lifecycle_msgs/srv/GetState`) a un délai de 2 s, puis on réessaie. Le contrôle de l'état du contexte ROS est refait à chaque tour, donc un arrêt demandé sort de la boucle.

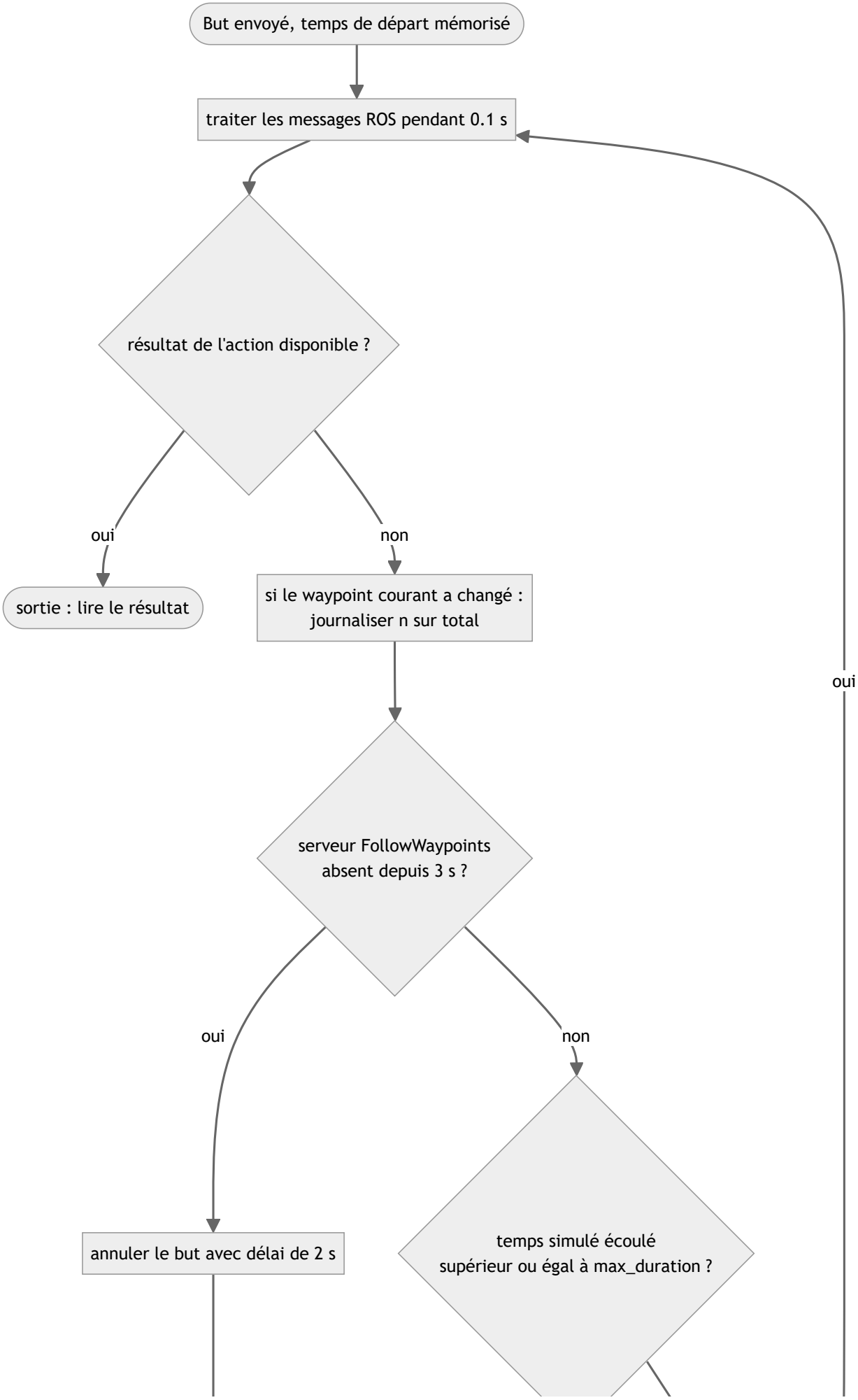

Début de l'attente
interroger /amcl/get_state
délai 2 s, nouvel essai si pas de réponse
non
amcl actif ?
oui
republier la pose initiale sur /initialpose
toutes les 1 s
non
/amcl_pose reçu ?
oui
interroger /bt_navigator/get_state
même règle de délai
non
arrêt de ROS demandé

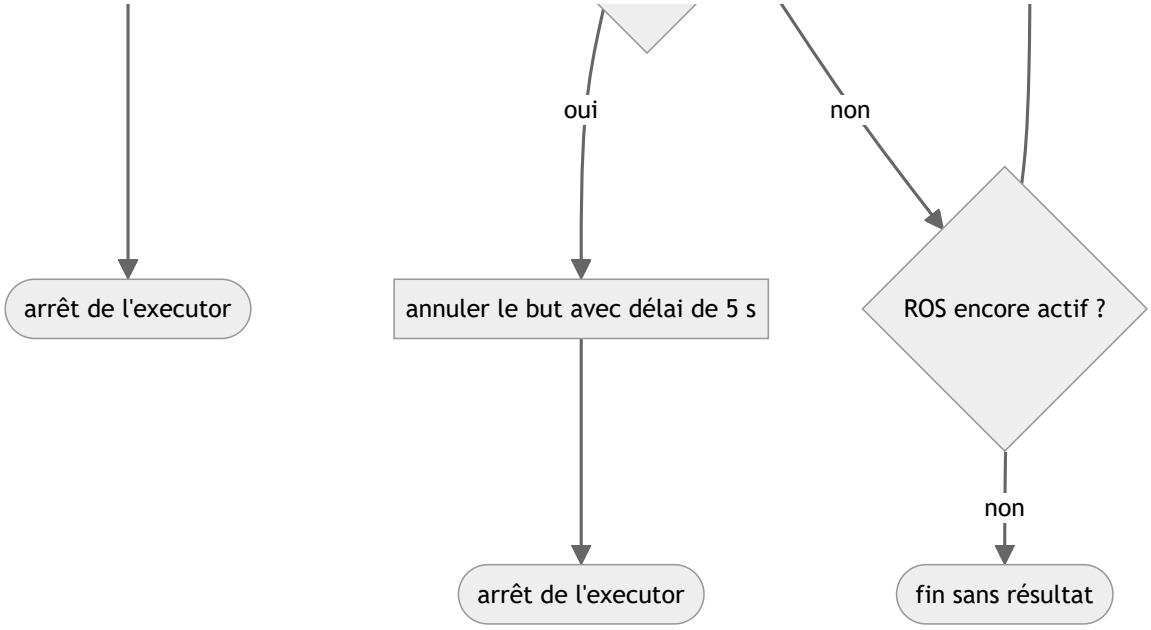

bt_navigator actif ?
oui
arrêt de ROS demandé
interroger /waypoint_follower/get_state
non
waypoint_follower actif ?
oui
serveur d'action FollowWaypoints
visible ? (délai 1 s par tour)
non
oui
arrêt de ROS demandé



- Ordre voulu : la pose initiale est republiée tant qu'AMCL n'a pas répondu, car chaque publication réinitialise le filtre ; l'intervalle d'environ 1 s laisse à AMCL le temps de répondre.
- `waypoint_follower` est activé en dernier par le lifecycle manager. Son serveur d'action doit être visible avant l'envoi, sinon le but est rejeté. C'est aussi pourquoi l'envoi autorise jusqu'à 5 essais.
- Raison de la réécriture : `BasicNavigator` n'a pas de délai d'attente. Un blocage a été observé sur 2 lancements sur 5, avec le message « amcl: failed to send response to /amcl/get_state (timeout) ». Voir JOURNAL, « Incident : Nav2 bloqué à l'attente d'activation » et « Correctif : blocage de l'executor au démarrage ».
- Limite connue : l'attente d'acceptation du but dans `followWaypoints` reste sans délai (bibliothèque). Correctif validé : 3 démarrages sur 3 conformes, puis les 3 runs officiels de B.

3.3 Boucle de suivi

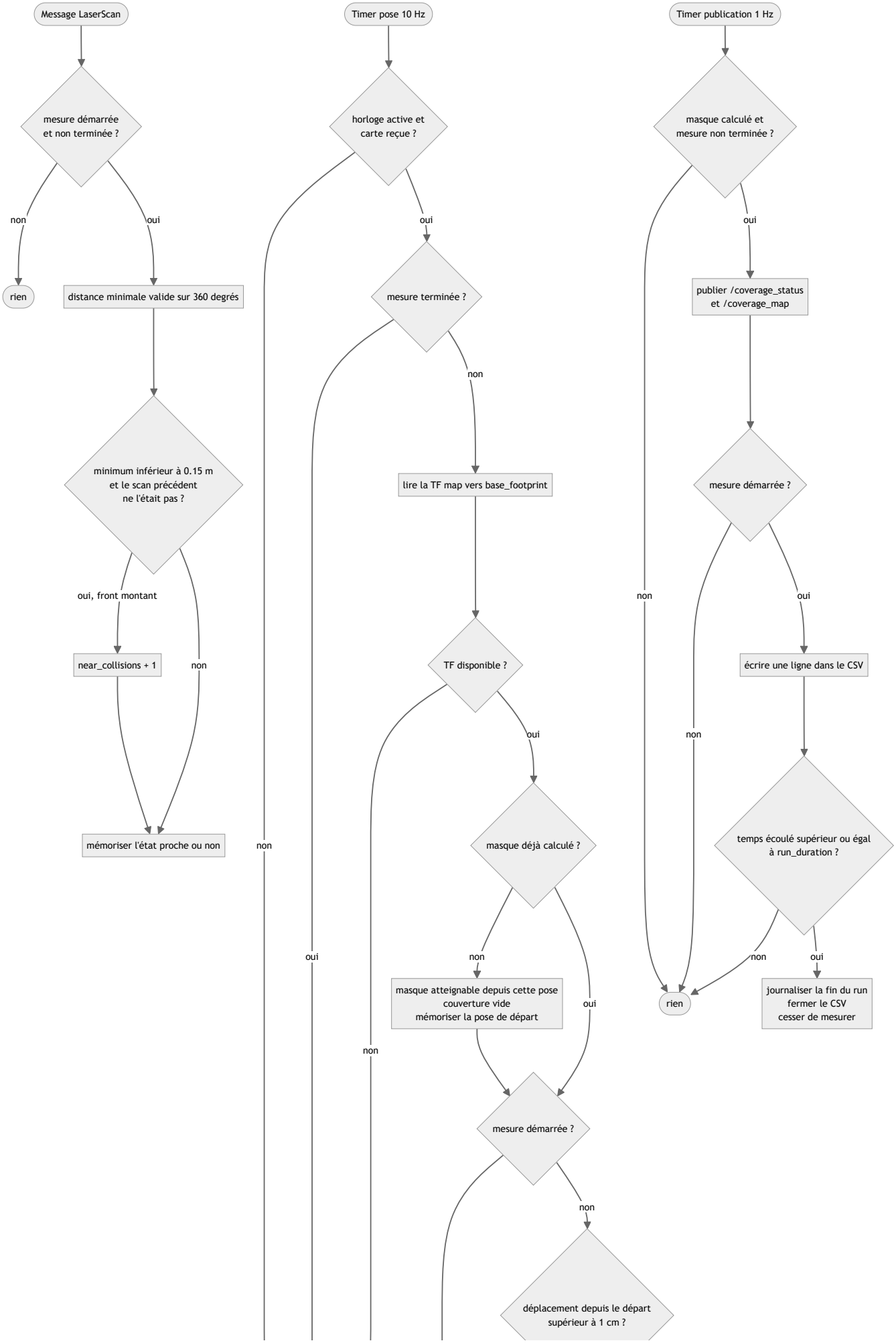

But envoyé, temps de départ mémorisé
traiter les messages ROS pendant 0.1 s
résultat de l'action disponible ?
oui
non
sortie : lire le résultat
si le waypoint courant a changé : journaliser n sur total
serveur FollowWaypoints absent depuis 3 s ?
oui
non
annuler le but avec délai de 2 s
temps simulé écoulé supérieur ou égal à max_duration ?
oui

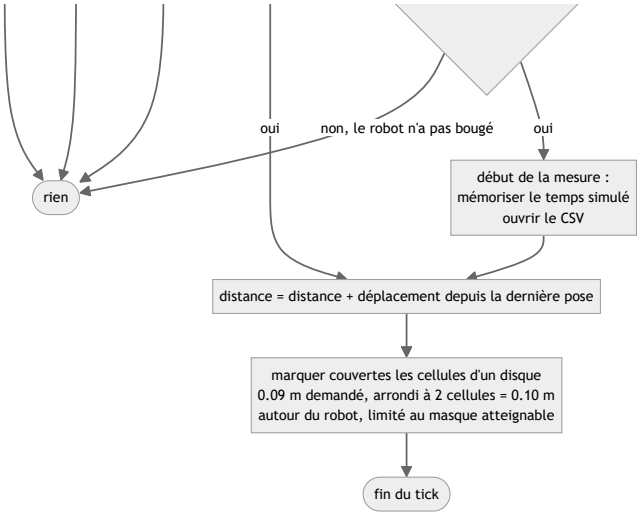


- Quatre sorties : fin du but, `max_duration` (= `run_duration` + 10 s, en temps simulé), perte du serveur pendant 3 s (mesurée en temps mural), arrêt de ROS ou signal d'arrêt (annulation du but).
- Le délai de 3 s évite de sortir à tort : la découverte DDS vacille parfois sous WSL.
- La boucle n'utilise pas `isTaskComplete()` : elle ne rendait jamais la main à l'arrêt de rclpy et fuyait en mémoire (4.8 Go). Voir JOURNAL, « Bug : fuite mémoire de coverage_executor ».
- Résultat : en cas de but terminé, les waypoints sautés (`missed_waypoints`) sont lus dans le résultat brut de l'action et journalisés (`stop_on_failure` est à false).

---

4. coverage_tracker (mesure commune A et B)


Message LaserScan
mesure démarrée et non terminée ?
non
oui
rien
distance minimale valide sur 360 degrés
minimum inférieur à 0.15 m et le scan précédent ne l'était pas ?
oui, front montant
non
near_collisions + 1
mémoriser l'état proche ou non
Timer pose 10 Hz
horloge active et carte reçue ?
non
oui
mesure terminée ?
oui
non
lire la TF map vers base_footprint
TF disponible ?
non
oui
masque déjà calculé ?
non
oui
masque atteignable depuis cette pose
couverture vide
mémoriser la pose de départ
mesure démarrée ?
non
déplacement depuis le départ supérieur à 1 cm ?
Timer publication 1 Hz
masque calculé et mesure non terminée ?
non
oui
publier /coverage_status et /coverage_map
mesure démarrée ?
non
oui
écrire une ligne dans le CSV
temps écoulé supérieur ou égal à run_duration ?
non
oui
rien
journaliser la fin du run
fermer le CSV
cesser de mesurer

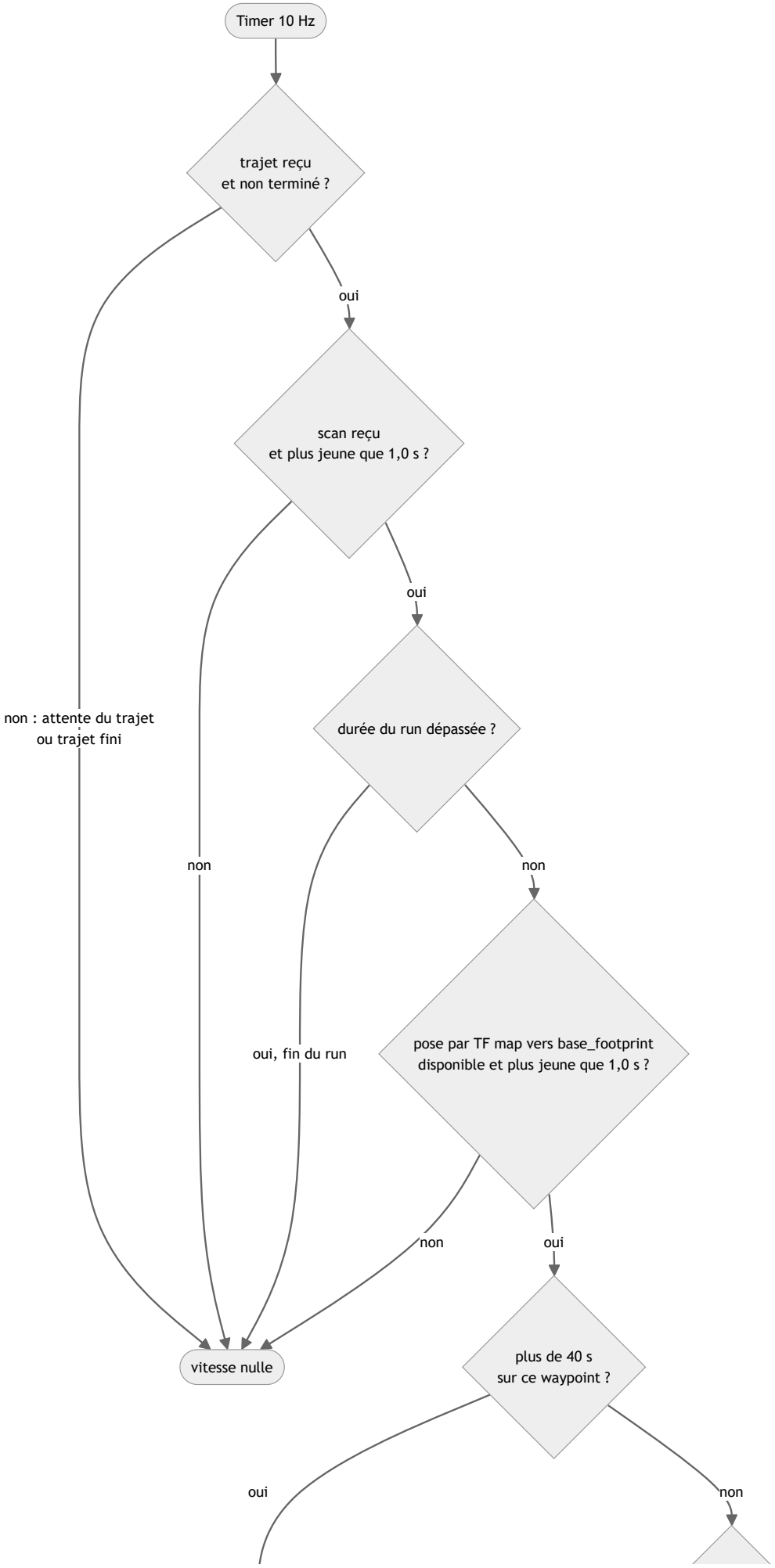


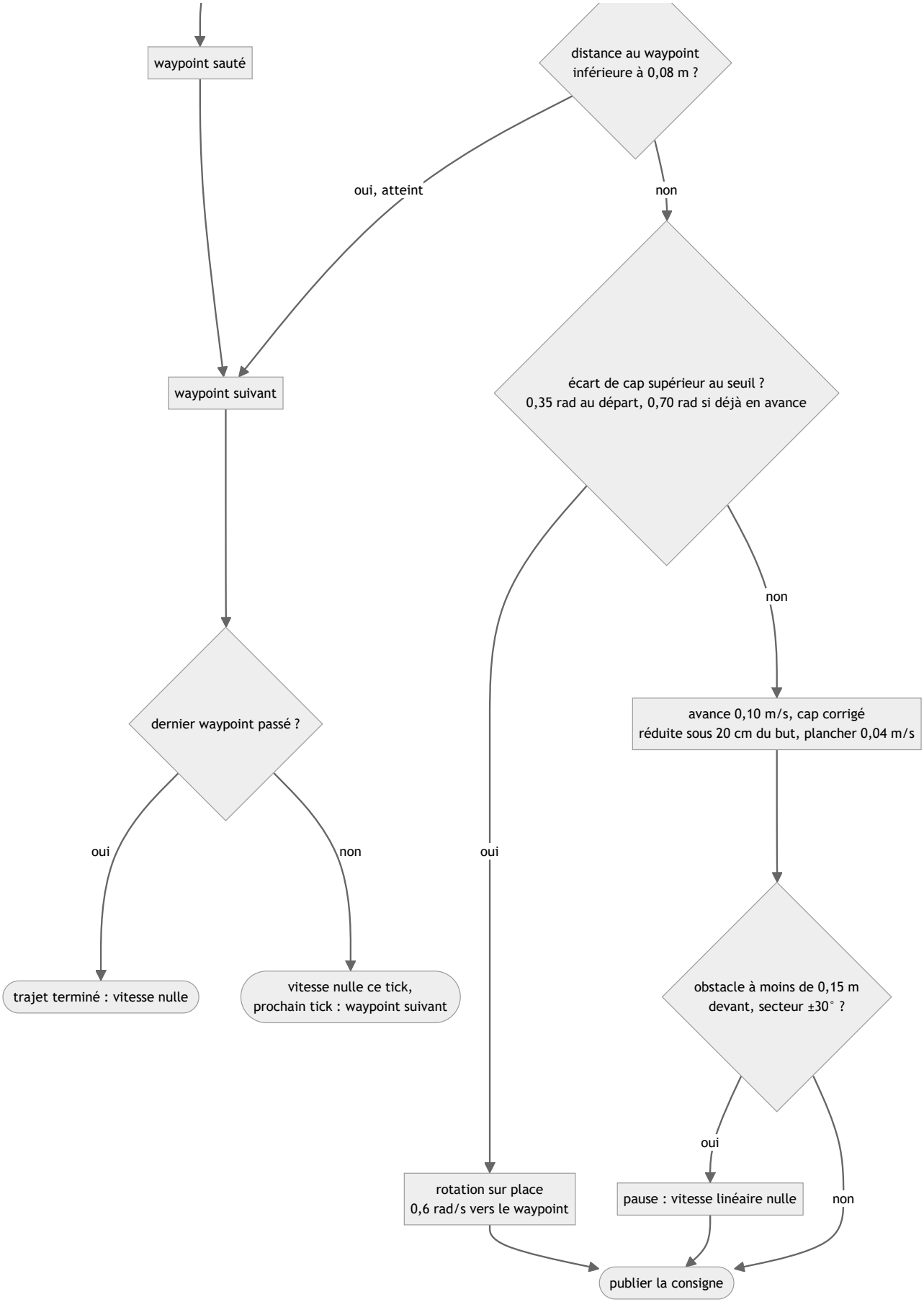
- Trois déclencheurs indépendants : la pose (10 Hz), le scan (5 Hz), la publication (1 Hz).
- Le pourcentage est le nombre de cellules couvertes dans le masque atteignable, divisé par la taille du masque.
- La mesure démarre au premier déplacement : le temps d'activation de Nav2 (stratégie B) n'est pas compté.
- La quasi-collision est comptée sur front montant : un passage sous le seuil compte pour un événement.
- Le seuil est de 0.15 m, car `range_min` du LiDAR (0.12 m) dépasse le rayon du robot (0.105 m).
- Le temps du run est le temps simulé.

---

5. coverage_follower (stratégie B sur le robot réel, sans Nav2)

Timer à 10 Hz. Chaque tick publie une consigne sur `/cmd_vel`.


Timer 10 Hz
trajet reçu
et non terminé ?
non : attente du trajet
ou trajet fini
oui
scan reçu
et plus jeune que 1,0 s ?
non
oui
durée du run dépassée ?
oui, fin du run
non
pose par TF map vers base_footprint
disponible et plus jeune que 1,0 s ?
non
oui
vitesse nulle
plus de 40 s
sur ce waypoint ?
oui
non



- Principe : « tourner puis avancer » (même idée que le `random_walker`). La logique de cap est dans `follow_logic.py`, testée sans ROS.
- Hystérésis : en avance, on ne repasse en rotation que si l'écart dépasse deux fois le seuil (0,70 rad). Cela évite l'alternance rotation/avance due à la latence Wi-Fi.

- Les arrêts de sécurité (scan ou TF absents ou périmés) publient une vitesse nulle sans faire avancer le trajet. Le délai de 40 s continue de courir pendant ces arrêts.
- L'obstacle ne met en pause que l'avance : la rotation sur place reste possible.
- Un waypoint atteint, sauté ou en fin de trajet remet la vitesse à zéro pour ce tick, puis le suivant démarre en rotation.